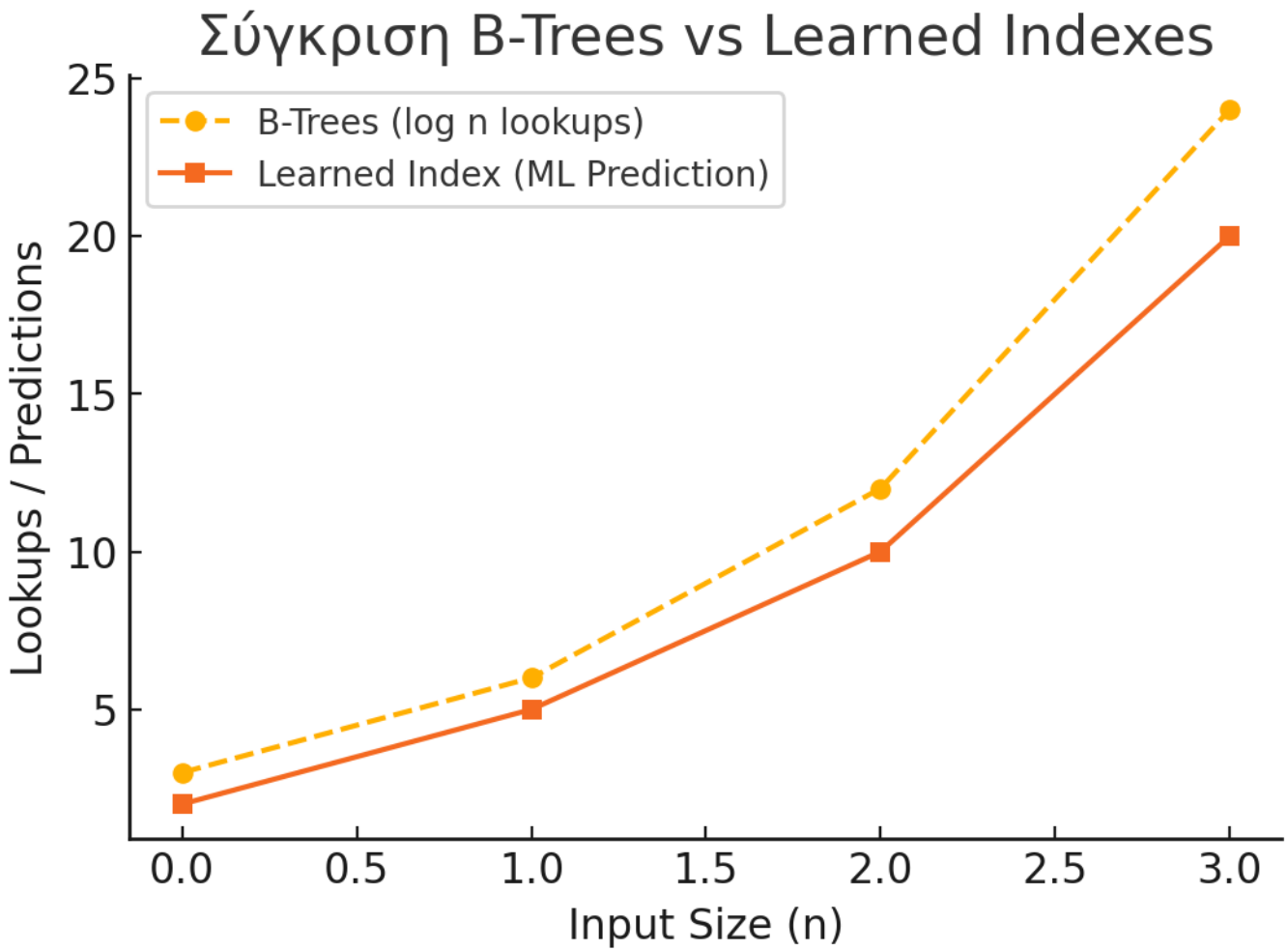
Learning Augmented Searching

Χρήση Machine Learning για
Βελτιστοποίηση Δομών Δεδομένων

LA Searching

- Χρήση Machine Learning για τη βελτιστοποίηση δομών δεδομένων
- Πρόβλεψη θέσης δεδομένων αντί για κλασικές τεχνικές ψαξίματος (LS,BS, IS, BIS, BIS*,BSTres, B-Trees, IST-trees e.t.c.)
- Στόχος: Ταχύτερες και αποδοτικότερες αναζητήσεις

Σύγκριση B-Trees vs Learned Indexes

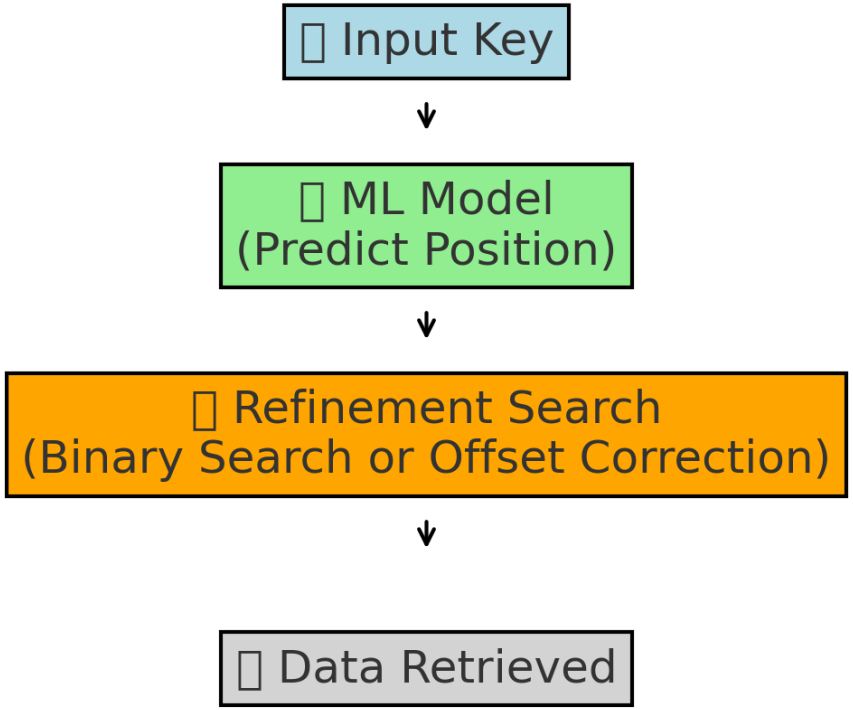


Πώς Λειτουργούν;

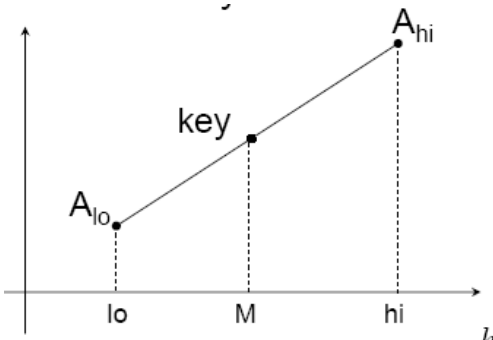
- Εκπαίδευση μοντέλου ML σε ταξινομημένα δεδομένα
- Το μοντέλο προβλέπει τη θέση του κλειδιού στη μνήμη
- Χρησιμοποιούνται επίπεδα μοντέλων για ακριβέστερη πρόβλεψη

Αρχιτεκτονική LA δομών

Βελτιωμένη Αρχιτεκτονική Learned Index



Interpolation Search



$$M \approx lo + \frac{hi - lo}{A_{hi} - A_{lo}} \times (key - A_{lo})$$

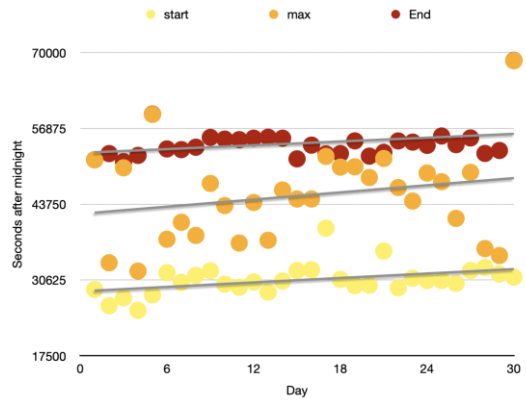
```
Let's assume that the elements of the array are linearly distributed.

General equation of line : y = m*x + c.
y is the value in the array and x is its index.

Now putting value of lo,hi and x in the equation
arr[hi] = m*hi+c ----(1)
arr[lo] = m*lo+c ----(2)
x = m*pos + c     ----(3)

m = (arr[hi] - arr[lo] )/ (hi - lo)

subtracting eqxn (2) from (3)
x - arr[lo] = m * (pos - lo)
lo + (x - arr[lo])/m = pos
pos = lo + (x - arr[lo]) *(hi - lo)/(arr[hi] - arr[lo])
```

Linear regression:

day	start	max	end
1	28970	51441	51420
2	26179	52499	33600
... data omitted for readability ...			
29	31660	53043	34860
30	31121	68629	68580

Table: input

Ενεργειακά Δεδομένα από φωτοβολταικα Panels (Kw/h).



x	y
x1	y1
x2	y2
……	……
x30	y30

x=day, y=start ή max ή end

The equation can be found in any statistical analytics text book: *y=m·x+b → key=m*pos+b → pos~(key-b)/m*

Where ***m* is called the slope** which determines the direction of the line and ***b* is called the intercept** which defines the distance of the line from 0.

$m=\sum(x_i - \ddot{x})(y_i - \ddot{y}) / \sum(x_i - \ddot{x})^2 = [(x1- \ddot{x})(y1- \ddot{y})+(x2- \ddot{x})(y2- \ddot{y})+\ldots+(x30- \ddot{x})(y30- \ddot{y})] / [(x1- \ddot{x})(x1- \ddot{x})+(x2- \ddot{x})(x2- \ddot{x})+\ldots+(x30- \ddot{x})(x30- \ddot{x}]$

$b= \ddot{y} - \cdot \ddot{x}\, m$, where $\ddot{x}$ and $\ddot{y}$ are the averages of x and y.

Μαθηματικό Μοντέλο (γραμμική παλινδρόμηση)

- Δινονται: (x1,y1),(x2,y2)…(xn,yn) (για εμάς y είναι η τιμή και x η θέση)
- y = mx+b (**ζητούμενο**)
- Γραμμική σχέση
- Η ευθεία προσεγγίζει τα δεδομένα

Η απλή γραμμική παλινδρόμηση είναι μια μέθοδος που:

- Περιγράφει τη σχέση μεταξύ **δύο μεταβλητών**
- Μίας ανεξάρτητης (x)
- Μίας εξαρτημένης (y)

Χρησιμοποιεί την μέθοδο ελαχίστων τετραγώνων.

Η απλή γραμμική παλινδρόμηση αποτυγχάνει όταν η σχέση των δεδομένων δεν είναι **γραμμική** (δηλαδή ακολουθεί καμπύλη), όταν υπάρχουν **ακραίες τιμές (outliers)** που παρασύρουν τη γραμμή, ή όταν η διασπορά των σφαλμάτων δεν είναι σταθερή (**ετεροσκεδαστικότητα**). Επίσης, καθίσταται αναξιόπιστη όταν οι παρατηρήσεις δεν είναι ανεξάρτητες μεταξύ τους (όπως σε χρονοσειρές με **αυτοσυσχέτιση**) ή όταν το φαινόμενο επηρεάζεται από άλλες σημαντικές μεταβλητές που έχουν παραλειφθεί, οδηγώντας σε μεροληπτικά αποτελέσματα. Ουσιαστικά, αν το διάγραμμα διασποράς των δεδομένων σας δεν θυμίζει ευθεία γραμμή με ομοιόμορφη κατανομή γύρω της, η απλή παλινδρόμηση θα σας δώσει μια παραπλανητική εικόνα της πραγματικότητας.

Υπολογισμός μέσων όρων και Πρόβλεψη ευθείας

- $\bar{x}$ = μέσος όρος x
- $\bar{y}$ = μέσος όρος y
- $m = \Sigma((x_i - \bar{x})(y_i - \bar{y})) / \Sigma((x_i - \bar{x})^2)$
- $b = \bar{y} - m\,\bar{x}$

```
avg(day)→day_bar,
avg(start)→ start_bar,
avg(max) → max_bar,
avg(end) → end_bar
```

day	day_bar	start	start_bar	max	max_bar	end	end_bar
1	15.5	28970	30647.0	51420	45272.0	51441	54322.8
2	15.5	26179	30647.0	33600	45272.0	52499	54322.8
3	15.5	27529	30647.0	50040	45272.0	51102	54322.8
…	…	…	…	…	…	…	…

Table: view1

Next we add the summaries as follows to calculate the slope:

```
sum((day - day_bar) * (start - start_bar)) /
        sum((day - day_bar) * (day - day_bar)) → start_slope,
sum((day - day_bar) * (max - max_bar)) /
        sum((day - day_bar) * (day - day_bar)) → max_slope,
sum((day - day_bar) * (end - end_bar)) /
        sum((day - day_bar) * (day - day_bar)) → end_slope
```

start_slope	max_slope	end_slope
128.292324805339	206.255839822024	109.764627363737

Table: view2

```
start_slope,
      maximum(start) - maximum(day) * start_slope → start_intercept,
max_slope,
      maximum(max) - maximum(day) * max_slope → max_intercept,
end_slope,
      maximum(end) - maximum(day) * end_slope → end_intercept
```

start_slope	start_intercept	max_slope	max_intercept	end_slope	end_intercept
128.292324805339	28658.4689655172	206.255839822024	42075.0344827586	109.764627363737	52621.4482758621

Table: view3

Predict the future!

With the slope and intercept for each series it's trivial to calculate the prediction for today or tomorrow.
The example uses 30 days history, so tomorrow is the 31st day.
We add the following outer query to the query from above:

```
start_slope * 31 + start_intercept → start,
max_slope * 31 + max_intercept →max,
end_slope * 31 + end_intercept → end
```

```
start_slope,
      maximum(start) - maximum(day) * start_slope → start_intercept,
max_slope,
      maximum(max) - maximum(day) * max_slope → max_intercept,
end_slope,
      maximum(end) - maximum(day) * end_slope → end_intercept
```

start_slope	start_intercept	max_slope	max_intercept	end_slope	end_intercept
128.292324805339	28658.4689655172	206.255839822024	42075.0344827586	109.764627363737	52621.4482758621

Table: view3

LA-searching Algorithm

Use Linear Regression and find which linear function fits the (X,Y) input, where X=pos AND y=KEY

Let y=m·x+b

Predict the position

*key=m*pos+b → pos~(key-b)/m*

Run LA-LS,LA-BS,LA-IS, LA-BIS, LA-BIS Around the pos*

Use 2 parallel threads and compare: (1) LS vs LA-LS, (2) BS vs LA-BS, (3) IS vs LA-IS, (4) BIS vs LA-BIS and (5) BIS vs LA-BIS**

Παράδειγμα Κώδικα (Python)

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Συνθετικά δεδομένα: ταξινομημένα κλειδιά και θέσεις στη μνήμη
keys = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100]).reshape(-1, 1)
positions = np.array([1, 5, 10, 15, 20, 30, 35, 45, 50, 60])  # Θέσεις των κλειδιών

# Εκπαίδευση του Linear Regression ως Learned Index
model = LinearRegression()
model.fit(keys, positions)

# Πρόβλεψη θέσης ενός νέου κλειδιού
query_key = np.array([[55]])  # Πού βρίσκεται το 55;
predicted_position = model.predict(query_key)

print(f"Το κλειδί {query_key[0][0]} εκτιμάται στη θέση: {predicted_position[0]:.2f}")
```

Παράδειγμα Κώδικα (Python)

```python
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

def learning_augmented_search(arr, target, model):
    """
    Εκτελεί αναζήτηση με τη βοήθεια ενός μοντέλου πρόβλεψης.
    """
    n = len(arr)

    # Χρησιμοποιούμε το μοντέλο για να προβλέψουμε τη θέση
    predicted_index = int(model.predict(np.array([[target]]))[0])
    predicted_index = max(0, min(predicted_index, n - 1))

    # Εκτελούμε τοπική δυαδική αναζήτηση γύρω από το σημείο
    left, right = max(0, predicted_index - 5), min(n - 1, predicted_index + 5)
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1  # Αν δεν βρεθεί

# Παράδειγμα χρήσης
np.random.seed(42)
n = 1000
arr = np.sort(np.random.randint(0, 10000, size=n))

```

Παράδειγμα Κώδικα (Python)

```python

# Δημιουργία δεδομένων για εκπαίδευση του μοντέλου
X_train = arr.reshape(-1, 1)  # Χαρακτηριστικά (τιμές στοιχείων)
y_train = np.arange(n)  # Θέσεις

# Εκπαίδευση ενός μοντέλου k-NN
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train, y_train)

# Δοκιμή της μεθόδου
target = arr[np.random.randint(0, n)]  # Τυχαίο στοιχείο από τη λίστα
index = learning_augmented_search(arr, target, model)
print(f"Το στοιχείο {target} βρίσκεται στη θέση {index}.")

```

Συμπεράσματα & Μελέτη

- Οι LA searching τεχνικές (Learned Indexes) μπορούν να μειώσουν δραστικά το κόστος αναζήτησης
- Εξαρτώνται από τη δομή των δεδομένων και το μοντέλο ML